7BUISO22W Cyber Security Applications

Web Exploitation and Mitigation of OWASP Vulnerabilities – SQL Injection (SQLi)

University Of Westminster

Dr. David Huang

Learning Objective

The objective of this lab is to equip students with practical, hands-on experience in actively exploiting common OWASP Top 10 vulnerabilities (A05:2025 – Injection), specifically SQL Injection, to bypass authentication mechanisms and to apply effective mitigation strategies to defend against SQL Injection attacks.

- If you are working on your own machine:
  - You must use Burp Suite Community Edition.
  - You must have DVWA and WebGoat running on your Kali Linux or DVWA on OWASP virtual machine. Ensure that both applications are up and running before starting the lab.
- If you are working on a CLG.43/45 machine:
  - Use the pre-installed Burp Suite Professional Edition and Docker software.
  - Ensure that DVWA and WebGoat are configured in Host-Only mode and running on your Kali Linux machine, with proper communication established with the OWASP virtual machine.

For your assignment, you need to set up the assignment scenario on Kali Linux.

Topics Covered

- String and Numeric SQL injection
- Blind SQL injection
- How SQL injection violates the CIA triad
- Mitigation and Best Practices for Defending Against SQL Injection

Note: Some of these vulnerabilities and tasks may apply to either OWASP DVWA or WebGoat.

Safety and Ethics Note

- This lab uses intentionally vulnerable systems solely for educational purposes.
- All activities must be conducted within the local lab virtual machine (VM) environment. Testing against external or production systems is strictly prohibited.

Lab Tasks

1. Lab Eenvironment Setup

In this lab, we will use two vulnerable web applications:

- **Damn Vulnerable Web Application (DVWA)**

  DVWA is a deliberately vulnerable web application designed for security testing practice. On the left-hand side of the interface, you will see a menu containing links to all the vulnerabilities available for practice within the application.

  You must use only the DVWA running on the OWASP Virtual Machine. Access DVWA via one of the following methods:

  http://192.168.56.101/dvwa/

  OR

  Navigate to http://192.168.56.101 and select Damn Vulnerable Web Application

  Login credentials (OWASP VM DVWA):

  - username**: admin**
  - password: **admin**

- **OWASP WebGoat**

  OWASP WebGoat is a deliberately vulnerable web application that covers several OWASP vulnerabilities through structured lessons and interactive challenges. It is designed to help students understand and assess their knowledge of common web security issues.

  The application is available as a Docker container on Kali Linux. The WebGoat Docker container is already running on port 8082.

  You can access WebGoat using one of the following methods:

  http://127.0.0.1:8082/WebGoat

  OR

  http://webgoat.local

  Login credentials (Docker WebGoat on Kali):

  - username**: webgoat**
  - password: **webgoat**

2. Pre-Lab Learning Activities: SQL Injection (Burp Suite Academy)

Before starting the lab session, students are required to complete the following preparation tasks from Burp Suite Academy:

1. Study SQL Injection (SQLi)

Visit the SQL Injection learning page: https://portswigger.net/web-security/sql-injection

Learn and understand:

- What SQL Injection (SQLi) is
- How to identify and exploit different types of SQLi vulnerabilities
- How to prevent SQL Injection attacks

2. Pre-Class Activities

Students must ensure they have an active account on PortSwigger.net. Registration is free and grants full access to the Web Security Academy.

Before attending the lab, review and understand the vulnerabilities covered in the SQL Injection section of Burp Suite Academy. Familiarize yourself with SQL Injection, its vulnerabilities, and types by completing the following related labs available at:

https://portswigger.net/web-security/all-labs#sql-injection

Such as:

- Retrieving hidden data, where you can modify a SQL query to return additional results.
- Subverting application logic, where you can change a query to interfere with the application's logic.
- UNION attacks, where you can retrieve data from different database tables.
- Blind SQL injection, where the results of a query you control are not returned in the application's responses.

These exercises will help you gain hands-on familiarity with SQL Injection techniques and prepare you for the practical lab activities using DVWA and WebGoat.

3. In class Lab Tasks: SQL Injection: Exploitation and Mitigation Using DVWA and WebGoat

- **SQL Injection – OWASP (DVWA & WebGoat)**

Part 1: Detecting SQL Injection Vulnerabilities

Determine whether the application is vulnerable to SQL Injection.

Procedure

1. Open DVWA on the OWASP virtual machine.
2. Navigate to **SQL Injection**.
3. Enter the following payload in the **User ID** field: '

Observation

- If an SQL error appears, the application is likely vulnerable.

Boolean Testing

Enter: ' OR '1'='1

Observation

- If the application returns additional records, the condition is being evaluated by the database.

Part 2: Retrieving Hidden Data

Retrieve unintended data from the database.

Procedure

1. Start Burp Suite.
2. Capture the request when submitting a User ID.
3. Send the request to **Repeater**.
4. Modify the parameter:

   1' OR '1'='1

Expected Result

- Multiple records are returned.

Part 3: Subverting Application Logic (Login Bypass)

Bypass the login mechanism using SQL Injection.

Procedure

1. Navigate to the login page (OWASP Bricks or DVWA login).
2. Attempt login with:

   Username: admin
   Password: ' OR '1'='1

   Alternative Method

   Username: admin'--
   Password: anything

Observation and Explanation

The -- operator comments out the rest of the SQL query, allowing authentication without verifying the password.

Part 4: SQL UNION Attacks

Determine the number of columns and retrieve database information.

Step 1: Identify Number of Columns

Test:

1' UNION SELECT 1,2 --

If successful, try:

1' UNION SELECT 1,2,3 –

Observation

- If no error occurs with two columns but fails with three, the query likely contains two columns.

Part 5: Retrieving Database Information

Identify Tables

```
1' UNION SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_name LIKE '%user%' --
```

Extract User Credentials

1' UNION SELECT user, password FROM dvwa.users --

Part 6: Blind SQL Injection (WebGoat)

Exploit Blind SQL Injection vulnerabilities.

Procedure

1. Open WebGoat.
2. Navigate to:
   Injection Flaws → Blind Numeric SQL Injection
3. Test:
   True condition: 101 AND 1=1
   False condition: 101 AND 1=2

Observation

Different responses confirm Blind SQL Injection vulnerability.

Part 7: Using Burp Intruder for Data Extraction

Automate extraction of sensitive information.

Procedure

1. Capture the POST request.
2. Send it to Intruder.
3. Insert payload:
   101 AND ((SELECT pin FROM pins WHERE cc_number='1111222233334444')=1000);
4. Add numeric payload range.
5. Use Grep Match to detect:
   - "Account number is valid"
   - "Invalid account number"
6. Start attack and identify the correct PIN.

- **How SQL Injection Violates the CIA Triad**

The Lab experiments demonstrate that improper input validation enables attackers to:

- **Manipulate SQL queries**, potentially altering data or application behaviour (violates **Integrity**).
- **Bypass authentication**, allowing unauthorized access to accounts (violates **Confidentiality**).
- **Retrieve confidential data**, exposing sensitive information such as usernames and passwords (violates **Confidentiality**).
- **Extract sensitive information through Blind SQL Injection**, potentially affecting database operations and availability (violates **Availability** and **Integrity**).

These activities highlight the severity of SQL Injection and its potential impact on the Confidentiality, Integrity, and Availability (CIA) of a system.

4. Post-Lab Questions

1. Why does ' OR '1'='1 bypass authentication?
2. What is the purpose of the -- operator?
3. How does UNION-based SQL Injection differ from Blind SQL Injection?
4. Why are prepared statements effective against SQL Injection?

5. Mitigation Techniques

To defend against SQL Injection, effective countermeasures include:

- Prepared Statements (Parameterized Queries).
- Input Validation and Sanitization.
- Principle of Least Privilege for database accounts.
- Suppression of detailed SQL error messages.
- Implementation of Web Application Firewalls (WAFs).

Lab Journal and Report Collection for Lab-Based Portfolio Assessment

You are required to prepare a lab report, including screenshots, that clearly describes what you have done and what you have observed. This report will help you build a lab journal for the lab-based portfolio assessment.